Software Engineering

UE24CS341A

Rehaan Jose Mathew
PES2UG24CS400
G Section

Lab 3 - Component Diagram

Architecture Selection

We chose Layered Architecture for the Personal Subscription & Recurring Expense Auditor System.

Architectural Choice

A three-layer style: a Presentation layer (Dashboard UI), a Business/Application layer (Import Service, Subscription Manager, Recurring Detection Engine, Notification Service, Auth Service) and a Data layer (encrypted Data Access Component). Layers communicate only through the provided/required interfaces shown in the component diagram (e.g. ISubscriptionMgmt, IDetectRecurring, ISubscriptionStore). Microservices were rejected because their operational complexity and network latency are unjustified for a single-user personal finance app; plain Client-Server was rejected because it gives no internal separation between parsing, detection and storage logic.

Two Reasons

Clear separation of concerns that maps to our requirements.

Each functional requirement lands in one layer: CSV/OFX import (FR-002) in the Import Service, recurring-pattern detection (FR-001) in the Detection Engine, burn-rate and cancellation guides (FR-004, FR-005) in the Subscription Manager and alerts (FR-003) in the Notification Service. A new file format such as another bank's OFX variant only changes the Import Service; the UI and database stay untouched.

The workflow is naturally sequential and top-down.

Our use cases follow one path: the user imports transactions, the system detects recurring charges, recalculates the burn rate, then displays and notifies. This request-response flow fits a layered pipeline, and the Subscription Manager acts as the single orchestrator, so the whole system can be built, tested and deployed as one unit in the three sprints we planned.

Security Advantage

The UI never talks to the database directly; every request must pass through the Business layer, and only the Data Access Component can touch stored data. This gives one choke point for enforcing NFR-001 and NFR-002: TLS 1.3 protects all traffic at the Presentation boundary, AES-256 encryption at rest is implemented once inside the Data layer, and the Auth Service checks roles (user vs. Finance Auditor) before any business operation runs. Raw banking credentials are never passed to the UI or stored in cleartext.

Performance Benefit

Business-layer components such as the Detection Engine and Subscription Manager run in the same process, so recurring-pattern analysis over a full transaction history involves in-memory calls instead of the network hops a microservices design would add. Burn-rate totals and renewal dates can be cached in the Subscription Manager and recalculated only when a subscription changes, keeping the dashboard responsive, while renewal alerts are handed to an asynchronous queue so notification delivery never blocks the user's request.

Trade-off Acknowledged

Layering adds some call overhead and couples each layer to the one below it, and the whole application must scale together. For one user's transaction history this cost is negligible, and the Data Access interfaces keep the database replaceable if the system grows

Personal Subscription & Recurring Expense Auditor — UML Component Diagram (Layered Architecture)

PRESENTATION LAYER

«component»

Dashboard UI Component

Web / mobile front end: My Subscriptions list, burn-rate dashboard, import wizard, cancellation guide, auditor report export, login

IImportTransactions

HTTPS / TLS 1.3 · CSV/OFX upload

ISubscriptionMgmt

HTTPS / TLS 1.3 · REST (JSON)

IAuthentication

HTTPS / TLS 1.3 · login & token

BUSINESS / APPLICATION LAYER

«component»

Import Service Component

Parses CSV / OFX uploads

Calls bank aggregation API

Validates & maps fields

(date, amount, merchant)

«component»

Subscription Manager Component

Core orchestrator for all use cases

Subscription list & renewal dates

Burn-rate calculation (monthly / annual)

Misclassification correction rules

Cancellation guide & auditor report export

IDetectRecurring

in-process call · transactions

«component»

Recurring Detection Engine

Finds recurring billing patterns

One-time vs subscription classifier

INotify

async queue · renewal events

«component»

Notification Service Component

Schedules alerts 5 days pre-renewal

Dispatches email / push messages

«component»

Auth Service Component

User & Finance Auditor login

Session / token management

Role-based access control

ITransactionStore

SQL over TLS · save transactions

ISubscriptionStore

SQL over TLS · read / write subscriptions

ICredentialStore

SQL over TLS · user & role lookup

DATA LAYER

«component»

Data Access Component (Encrypted Repository)

Transactions · subscriptions · merchant rules · user profiles · credential hashes— AES-256 at rest

Provided interface (ball)

Required interface (socket)

Assembly connector = socket + ball (requirer → provider).
Labels show interface name and protocol.